Interfaces

Monday, 31 August 2026 9:14 AM

Interfaces allow you to focus on what a type does rather than how it's built. They can help you write more flexible and reusable code by defining behaviors (like methods) the different type can share. This makes it easy to swap out or update parts of your code without changing everything else.

Interfaces are just collections of method signatures. A type implements an interface if it has method that match the interface's method signatures.

+ Interface Implementation-:

Interface are implemented implicitly. A type never declares that it implement a given interface. if an interface exists and a type has the proper methods defined, then the type automatically fullfills that interface.

- A type implements an interface by implementing its methods. Unlike in many other languages, there is no explicit declaration of intent. there is no "implements" keyword.
- Interfaces are collections of method signatures. A type "implements" an interface if it has all of the methods of the given interface defined on it.

* Multiple Interfaces-:

A type can implement any number of interfaces in GO. e.g. the empty interface, interface{}. is always implemented by every type because it has no requirements.

- you can nam your interface parameters

```
type Copier interface{
    Copy (sourcefile string, destinationfile string)(bytecopied int)
}
```

+ Type Assertions in GO

When working with interfaces in Go, every once-in-awhile you'll need access to the underlying type of an interface value. you can cast an interface to it's underlying type using a type assertions.

+ Type Switches

A type switch makes it easy to do several type assertions in a series. A type switch is similar to a regular switch statement, but the cases specify types instead of values.

* Clean Interface-:

1. Keep Interface Small

if there is only one piece of advice that you take away from this lesson, make it this: keep interface small!! Interfaces are meant to define the minimal behavior necessary to accurately represent an idea or concept.

2 Interfaces should Have No Knowledge of Satisfying Types-:

An interface should define what is necessary for other types to classify as a member of that interface. They shouldn't be aware of any types that happen to satisfy the interface at design time.

3. Interfaces Are Not Classes-:

- Interfaces are not classes, they are slimmer.
- Interfaces don't have constructors or destructors that require that data is created or destroyed.
- Interface aren't hierarchical by nature. though there is syntactic sugar to create interfaces that happen to be supersets of other interfaces.
- Interfaces define function signatures, but not underlying behavior. Making an interface often won't DRY up your code in regard to struct methods.